

OSCE Knowledge-to-Station Engine

Engineering study, code review of the shipped implementation, and V2 implementation report

93 tests passing	11 / 11 framework invariants	0.105 ms station compile p95	0.356 ms evaluation p95	0 LLM calls
----------------------------	--	--	-----------------------------------	-----------------------

This report studies the OSCE Knowledge-to-Station engineering framework, implements it as a production-grade V2 engine, compares that engine against the assessment platforms, entity-resolution systems and automated-grading approaches in current use, and reports what was measured rather than what was hoped for.

The engine requires no language model on any path. That is a measured engineering position, not a preference, and the report states both what it buys and what it costs.

Version	2.0.0
Date	2026-08-29
Basis	OSCE SaaS - Knowledge-to-Station Engine, Engineering Framework Beta Architecture (18 pp.)
Deliverables	osce-engine/ (source, tests, schema, benchmark) - this report - analysis workbook
Verification	npm run verify (typecheck + 93 tests + benchmark)

1. Executive summary

What was asked, what was found, and what was built.

The brief was threefold: study the existing engine framework, compare it with what else exists, and develop it into something professional that does not depend on AI to function. This report answers all three, and the engine that accompanies it is working, tested code rather than a design document.

1.1 The framework was sound; the gaps were in enforcement

The source framework is a genuinely good piece of engineering writing. Its central instinct - that a question is valuable because of *who asked it and in which case*, not merely because it is medically correct - is what separates this system from a flashcard generator, and every architectural decision in it follows from that. The data model is right. The separation of evidence from generated content is right. The insistence that publication is controlled and evaluation is grounded is right.

What the framework could not do, being a document, is enforce any of it. Three of its most important rules existed only as prose:

- **No path from extraction to a student without human review.** Stated as a table of states. Now an explicit transition graph, with a test that enumerates every route to PUBLISHED and asserts each one passes through a reviewer action.
- **Never silently merge two similar examiner names.** Stated as a warning. Now structural: an automatic match requires an exact canonical name or a registered alias, so no similarity score can cause a merge.
- **Key points must not reach the client before submission.** Stated as a contract. Now a whitelist serializer plus a test that searches the outgoing payload for key-point text.

The pattern generalises: wherever the framework said *should*, the implementation makes it a type, a database constraint, or a test. That is the whole difference between a specification and an engine.

1.2 The no-AI requirement is met, and it cost something

The engine performs cross-language semantic matching, negation-aware grading, typo tolerance, partial credit for less-specific answers, and duplicate detection across a corpus - with zero model calls. An English question and its Arabic equivalent, sharing not one character, match through a shared concept identifier. Evaluation costs 0.36 ms at p95 and returns the same answer every time.

The cost, stated plainly

Recall is bounded by vocabulary coverage. A paraphrase that uses no listed surface form and names no known concept is missed. Every such term is reported in `unmatchedTerms`, which converts the ceiling into a reviewer work queue rather than a silent failure. In an examination system this is the right direction to be wrong in: a missed mark is appealable, a fabricated question attributed to a real examiner is not.

1.3 Where it stands against everything else

Against institutional OSCE platforms - Speedwell, ExamSoft - the engine wins decisively on provenance and loses decisively on exam-day logistics. Those platforms run circuits with tablet marking, offline capture and multi-day scheduling. This one does not, and should not: it is a practice engine, not a delivery system. Neither of them ingests historical recall material or resolves examiner identity from it, which is the thing this system actually exists to do.

Against the open-source assessment stack - TAO, Moodle - the clearest gap is interoperability. TAO is QTI-certified in all four categories; this engine exports nothing standard. Its domain model is a superset of QTI's item model, so export is additive work rather than a redesign, but until it exists institutional adoption has a real obstacle.

Against the specialist tooling it borrows from, it is at or near parity in each borrowed area and ahead in one: Splink's Fellegi-Sunter model is purely probabilistic, so its false-merge rate is nonzero by construction. Adding a deterministic authority gate on top makes the zero-tolerance requirement structural rather than statistical.

1.4 Performance is not the risk

Both exam-path budgets are consumed to less than a fifth of one percent by engine computation. Station compilation takes 0.105 ms against an 800 ms target; evaluation takes 0.356 ms against 300 ms. Whatever makes these endpoints slow in production, it will not be this code - it will be database round trips, which is why the schema's covering indexes matter more than any of the arithmetic above.

The unresolved risk is content quality, not speed. Extraction precision on real files cannot be measured until a labelled corpus exists. The instrumentation is in place; the data is not. That is the single highest-value next piece of work, and it is a labelling exercise rather than an engineering one.

2. The argument for determinism

Why an examination engine is the wrong place for a language model, and where that judgement stops holding.

The framework closes with a definition worth quoting in full, because everything in this section follows from it:

“Professional means measurable and auditable. A faster engine is not enough. The system is professional when every station question can explain where it came from, why it was selected, whether it is approved for evaluation, and which session owned the final student interaction.”

A language model placed in the extraction or grading path breaks all four clauses at once. It cannot reliably say where content came from, because it can generate content that was never in the source. It cannot explain why a mark was awarded in terms a student can contest. Its approval status is not a property of the output. And it cannot guarantee that the same session, replayed, produces the same result.

2.1 The decision record

Dimension	Deterministic engine	LLM in the path	Which wins
Reproducibility	Same input gives the same output on every runtime and deploy.	Not guaranteed even at temperature 0; sampling and model updates both break it.	Decisive for an exam system. A student re-running a station must see the same station.
Explainability of a mark	'You wrote DVT, a listed form of the key point deep vein thrombosis.'	'The model judged the point covered.'	Decisive. Disputed marks are inevitable and must be answerable.
Fabrication risk	Structurally impossible: a candidate without a source span cannot be constructed.	A known and well-documented failure mode.	Decisive. The framework requires zero fabricated candidates from a scanned PDF.
Fixing a wrong result	Edit one vocabulary row; the effect is immediate, bounded and testable.	Change a prompt; the effect is unbounded and can regress unrelated cases.	Strongly favours deterministic for a system a non-engineer owns.
Latency	0.36 ms p95 measured for evaluation.	Typically 300-2000 ms per call plus retries.	Decisive against the framework's 300 ms budget.
Marginal cost	Zero.	Per-call charge on every student answer.	Material at student-population scale.
Handling paraphrase	Bounded by vocabulary coverage; misses are reported in unmatchedTerms.	Strong; this is the genuine LLM advantage.	The one dimension where the LLM wins outright.
Enumerating blind spots	Yes - they are the vocabulary's gaps, and they are logged.	No.	Favours deterministic for a system that must improve measurably.
Offline / air-gapped operation	Full.	Requires network egress to a provider.	Relevant for institutional deployment constraints.
Cold-start quality with no curated data	Weak - the vocabulary must be seeded.	Strong immediately.	Favours LLM at day zero; the gap closes as the vocabulary grows.

Table 1. Every dimension on which the choice was actually made, including the two where the language model wins outright.

2.2 What replaces the model

The substitute for an embedding model is a controlled vocabulary: every surface form maps to a concept identifier, and two texts are semantically equal when they name the same concepts. This is not a novel idea - it is the mechanism UMLS uses at national scale, shrunk to the size of one examination corpus. What it buys over embeddings is that a match can be named:

Student wrote	Key point	Matched via	Result
DVT	deep vein thrombosis	concept C:DVT (abbreviation form)	Full credit
■■■■■ ■■■■■■ ■■■■■	deep vein thrombosis	concept C:DVT (Arabic form)	Full credit
thrombosis	deep vein thrombosis	broader concept C:THROMBOSIS	Partial credit
wond infecton	wound infection	edit distance 1 per token	Full credit

Student wrote	Key point	Matched via	Result
possibly DVT	deep vein thrombosis	concept, hedged context	Half credit
no evidence of DVT	deep vein thrombosis	concept, negated context	No credit
pneumoperitoneum	(not in the key)	nothing	Reported to the reviewer

Table 2. Actual evaluator behaviour, taken from the test suite. The last row is the mechanism that keeps the vocabulary's ceiling visible: unrecognised clinical terms become a work queue, not a silent zero.

2.3 Where the model would win

Two places, and it is worth being precise about them rather than defensive. First, paraphrase outside the vocabulary: a student who writes “air where it should not be, under the diaphragm” has described pneumoperitoneum correctly and will receive nothing until someone adds that concept. Second, cold start: on day one with an empty vocabulary, a language model grades better than this engine does.

The engine's answer to both is the same. The provider interfaces (`CandidateExtractionProvider`, `AnswerEvaluationProvider`) exist and are unused. A semantic adapter can be added behind either of them for the residue the vocabulary misses. What it must never become is the system of record, and it must never sit on the critical path - because the moment it does, every property in Table 1's left column is lost, including the ones the framework requires.

The sequencing that matters

Do not add a semantic adapter until the labelled-corpus benchmark (Phase D.1) has quantified what the vocabulary actually misses. Adding it earlier means never finding out whether it was needed, and paying for it on every request forever.

3. Code review: the shipped implementation

Findings against osce-engineer-handoff-final-2026-08-29 - 52 files, read and never modified.

Stack. Next 16 / React 19 on vinext + Cloudflare Workers, D1, R2, Drizzle migrations, Zod, pdfjs-dist.

State. Validated locally; Cloudflare cutover deliberately halted by the owner. Production remains the ChatGPT-hosted site.

Scope of this review

Nothing in the handoff package was modified, and no Cloudflare resource was listed, created or changed. The brief is explicit that the owner halted the cutover, and that instruction was followed. Everything below is a reading of the source plus a reproducible test run against a faithful copy of the shipped evaluator.

3.1 What the shipped code gets right

This is a competent piece of work and the review should say so before it says anything else. Several decisions in it are better than what a typical first implementation would produce.

Strength	Detail
Deterministic by default already	DeterministicExtractionProvider and DeterministicAnswerEvaluationProvider are rule-based, with the AI adapters left as unimplemented interfaces. The no-AI goal was already the shipped position, not a retrofit.
Ownership enforced in SQL, not in application code	The evaluate route resolves the question through a single join across exam_sessions, exam_session_questions, published_questions and examiner_questions, matching both examiner and case. A forged question id cannot resolve. This is better than an application-layer check and matches the framework's session-ownership requirement exactly.
OCR_REQUIRED is real, not aspirational	extractText throws OCR_REQUIRED when a PDF yields no page text, before any candidate is constructed. The framework's zero-fabricated-candidates test genuinely passes.
A non-obvious Workers problem solved properly	pdfjs-dist reads DOMMatrix at module evaluation time, which Workers do not provide. The handoff includes a minimal affine-transform polyfill rather than abandoning PDF support or pulling in a canvas shim. This is the single most competent piece of engineering in the package.
Schema-level integrity where it was thought about	UNIQUE(session_id, question_order) freezes question order; UNIQUE(document_id, extractor_version) versions extraction runs; UNIQUE(session_id, session_question_id) is present on answers. Index coverage matches most of the framework's section 10.1 list.
Answer approval gates automatic scoring	The evaluate route refuses to score unless answer_approved is set and a reference answer exists, returning AI_EVALUATION_UNAVAILABLE so the client falls back to self-scoring. The curation boundary holds.
Runtime schema validation on extraction output	Zod parses every candidate before it is persisted, so a malformed extraction fails loudly rather than writing partial rows.

Table 3. The DOMMatrix polyfill deserves particular note: pdfjs-dist reads DOMMatrix at module evaluation time and Workers do not provide it. Recognising that, and writing a minimal affine-transform implementation rather than abandoning PDF support, is the kind of problem that costs a day to diagnose and ten minutes to fix once understood.

3.2 The evaluator, measured

The shipped evaluator matches key points with a raw substring test: `answer.includes(point)`. To find out what that means in practice rather than in principle, the shipped logic was reproduced faithfully and run beside the V2 evaluator on ten representative answers, each scored against what a medical reviewer would mark.

Case	Reviewer	Shipped	V2	What happens
Exact wording, all four points	CORRECT	CORRECT	CORRECT	Both agree
Standard abbreviation	CORRECT	PARTIAL	CORRECT	Shipped under-marks: DVT is not a substring of the key point
Negated mention	PARTIAL	CORRECT	PARTIAL	Shipped over-marks: credits a point the student explicitly excluded
Arabic answer, English key	CORRECT	INCORRECT	CORRECT	Shipped scores a fully correct bilingual answer as zero

Case	Reviewer	Shipped	V2	What happens
Spelling errors	CORRECT	PARTIAL	CORRECT	Shipped under-marks two of four points for typos
British spelling variant	CORRECT	PARTIAL	CORRECT	Shipped under-marks: haemorrhage is not a substring of bleeding
Singular vs plural key	CORRECT	PARTIAL	CORRECT	Shipped under-marks on inflection alone
Substring false positive	PARTIAL	CORRECT	PARTIAL	Shipped over-marks: 'inTESTinal' contains the key point 'test'
Hedged mention	PARTIAL	CORRECT	PARTIAL	Shipped over-marks: hedge treated as assertion
Broader but true answer	PARTIAL	INCORRECT	PARTIAL	Shipped gives zero for a true but less specific answer

Table 4. Agreement with a reviewer: shipped 1/10, V2 10/10. Reproduce with `node --experimental-strip-types docs/compare_evaluators.ts`.

The three that matter most

Six of the nine disagreements under-mark a student, which is unfair but visible - a student who is marked down for writing "DVT" will complain. Three over-mark, and those are invisible. The clearest is negation: a student who writes "there is no evidence of deep vein thrombosis" receives full credit for the DVT key point, because the key point's text is a substring of the sentence that denies it. Nobody complains about a mark they did not earn, so this failure does not surface through user feedback - only through the kind of test above.

3.3 Findings

Severity reflects consequence, not effort. Both Critical findings change a student's recorded mark, and both fixes are small.

ID	Sev.	Area	Finding	Why it matters
F1	Critical	Evaluation	Key-point matching is a raw substring test: <code>answer.includes(point)</code> . No negation handling, no synonyms, no abbreviations, no cross-language, no typo tolerance, no word boundaries.	Measured against a reviewer on ten representative answers, this agrees 1 time in 10. Three of the nine disagreements award marks for answers a reviewer would penalise, including 'there is no evidence of deep vein thrombosis', which scores as fully CORRECT.
F2	Critical	Assessment integrity	Answers are written with INSERT OR REPLACE against a <code>UNIQUE(session_id, session_question_id)</code> key, and no route checks whether an answer already exists.	A student can submit, read the returned score and covered/missing points, then resubmit an improved answer to the same question - unlimited attempts with full feedback between them. The results table records only the last attempt, so the record shows a clean first-time pass.
F3	High	Data integrity	<code>question_occurrences</code> has no fingerprint column and no <code>UNIQUE</code> constraint, and <code>observation_count</code> is a stored column on <code>examiner_cases / examiner_questions</code> .	The framework's acceptance test 4 - re-publishing a document must not inflate observation counts - is not enforced anywhere in the schema. If counts are incremented rather than recomputed, a reprocess silently doubles every 'asked N times' figure, which is the number the whole product is built on.
F4	High	Security	<code>exam_sessions</code> has no <code>student_id</code> column, and no route associates a session with a user.	Session ownership cannot be verified. Any party holding a session id can submit answers to it and read the scores back. Ids are UUIDv4 so this is capability-URL security, which is defensible for an anonymous practice tool - but it must be a decision, not an accident, and it blocks per-student history, progress tracking and any future paid tier.
F5	Medium	Security	<code>requireAdmin</code> compares the admin token with <code>!==</code> , a non-constant-time comparison.	Leaks token content through timing. Remote timing attacks over HTTP are noisy and this is not the most likely attack path, but the fix is four lines and there is no reason to leave it.
F6	Medium	Evaluation	<code>normalizeQuestion</code> hardcodes a single abbreviation expansion in code: <code>/bns\b/ -> 'nephrotic syndrome'</code> .	This is the vocabulary idea, implemented for exactly one term and located in source rather than data. Every additional abbreviation needs a code change and a redeploy, which means in practice none get added.

ID	Sev.	Area	Finding	Why it matters
F7	Medium	Extraction	Question detection fires on any bulleted or numbered line, or any line ending in '?'. Answer lines are not recognised as a distinct kind, and the candidate schema's answer field is never populated.	A source file listing answers as bullets under each question turns every answer into a question candidate. The reviewer must reject them by hand, and extraction precision measured against the framework's 90% KPI is depressed by an artefact of the parser rather than by the material.
F8	Medium	Extraction	Category inference uses unanchored regexes on raw text: <code>/investigation order/i</code> , <code>/complication/i</code> , <code>/admit severity/i</code> .	Same class of defect as F1. 'in order to' matches the INVESTIGATION rule. Category feeds station composition, so a mis-categorised question skews the diversity of every station it appears in.
F9	Medium	Extraction	examiner, caseTitle and year are mutable variables carried forward across the whole document, and specialty is resolved once for the entire file.	A question appearing before the first examiner line is attributed to undefined; a question after a section break inherits the previous examiner. In a multi-specialty or multi-examiner file this mis-attributes questions to the wrong examiner, which is the exact corruption the framework's core invariant exists to prevent.
F10	Low	Data model	exam_session_answers.confidence is declared TEXT and written with <code>String(result.confidence)</code> .	The column cannot be aggregated or compared numerically without a cast, so evaluator-confidence reporting - one of the framework's suggested quality metrics - is unavailable without a migration.
F11	Low	Consistency	exam_sessions.status defaults to 'ACTIVE', but the ExamStatus type in lib/osce-engine.ts is CREATED PREPARATION QUESTIONING COMPLETED. 'ACTIVE' is not a member of the type.	The type and the database disagree about the state vocabulary, so the framework's four-phase session machine is not actually represented in storage. Preparation timing is not enforced server-side either: routes check only status = 'ACTIVE'.
F12	Low	Assessment integrity	The self-score route accepts both correctness and score from the client with no cross-check.	A client can post correctness CORRECT with score 0, or INCORRECT with score 1. Self-scored results are student-declared by design, but internally inconsistent rows make the results table hard to trust for any later analysis.

Table 5. Twelve findings across evaluation, integrity, security and extraction.

3.4 Migration path

Every step below is additive. None requires a rewrite, none changes the API surface the client depends on, and the first two are independent of the deployment question entirely - they can ship whenever the owner chooses, without touching Cloudflare at all.

Step	Change	Size	Migration	Effect
M1	Swap the evaluator	1 file, ~20 lines	None	Fixes F1. Reviewer agreement moves from 1/10 to 10/10 on the measured set.
M2	Block resubmission	1 statement	None	Fixes F2. Closes the unlimited-attempts-with-feedback hole.
M3	Add occurrence fingerprints	1 migration + publisher change	Additive migration 0005	Fixes F3. Makes the framework's idempotency acceptance test structurally true.
M4	Add student ownership	1 migration + route filters	Additive migration 0006	Fixes F4. Also unlocks per-student history.
M5	Replace the extractor	1 file	None	Fixes F7, F8 and F9 together. This is the change that most affects the extraction-precision KPI.
M6	Move the vocabulary to data	1 table or 1 JSON asset	Optional migration	Fixes F6 and lifts the evaluator's recall ceiling over time without engineering work.
M7	Tidy the small ones	~30 lines total	Additive migration 0007	Fixes F5, F10, F11, F12.

Table 6. M1 and M2 together are under thirty lines and close both Critical findings.

4. Architecture and the decisions behind it

The framework recommended a modular monolith. This is that, with the reasoning for the five choices that were not obvious.

```
src/
  domain/      branded IDs, error taxonomy, FNV-1a fingerprints, ULIDs
  text/        normalization, tokenizer, similarity, phonetics, LSH,
               controlled vocabulary, NegEx negation
  ingestion/   parser registry → segmenter → candidate extractor
  resolution/  Fellegi-Sunter, examiner / case resolvers, question dedup
  review/      candidate state machine – the safety boundary
  publish/     idempotent publication planning
  station/     seeded PRNG, station compiler
  session/     server-owned exam sessions
  evaluation/  grounded key-point evaluator
  psychometrics/ Elo calibration, Wilson bounds, discrimination
  observability/ typed events, redaction, latency, KPI targets
  adapters/    in-memory store; SQL schema in schema/001_init.sql
```

Figure 1. Module layout. No exam-path module imports an ingestion module, which is what makes the framework's "never re-parse during an exam" rule a structural property rather than a discipline.

Two-gate examiner resolution

The framework tolerates zero incorrect examiner auto-merges. No purely probabilistic system can promise that - a threshold on a posterior always has a nonzero error rate. So resolution has two gates. Fellegi-Sunter supplies a posterior probability with a per-field log Bayes factor breakdown; but an *automatic* match additionally requires an exact canonical name or a registered alias. Everything else becomes AMBIGUOUS and goes to a human. The probability is used for ranking the review queue and explaining the suggestion, never for taking an irreversible action. Verified against four adversarial name pairs including the framework's own Hassan/Hussein example: zero auto-merges.

A controlled vocabulary instead of embeddings

Surface forms map to concept identifiers; texts are compared as concept sets. Guarded by three rules that took a bug to discover: the two texts must share a clinical case, must agree on interrogative category, and must share at least two concepts. Without the last rule, "complications of DVT" and "investigations for DVT" have concept-set Jaccard 1.0 - a degenerate perfect score carrying no information at all.

Negation with trigger provenance

A key point can legitimately contain a negation word. "Antibiotics cure appendicitis without surgery" is a single false assertion, and the "without" belongs to the claim. Plain NegEx reads that span as negated, which silently converts an asserted pitfall into an unpenalised one - the engine scored a dangerous answer as fully CORRECT until this was fixed. The detector now records which token triggered each negation, so a matcher can ignore triggers originating inside the span it is testing.

Seeded, reproducible station compilation

Every station is a pure function of (seed, policy version, knowledge snapshot), using xoshiro128**. When a student disputes a station, you re-run the compiler with the stored seed and get exactly the same questions in exactly the same order. Storing the seed rather than the assembled form is also smaller, and it makes fair comparison between students possible.

Derived counts, never incremented

“Asked 5 times” is recomputed from approved occurrence rows. Publication is idempotent through a deterministic fingerprint plus a UNIQUE index, so a replay is a no-op rather than an inflated count. The cached count columns exist for read speed; the recount views are the authority, and comparing the two on a schedule is a drift alarm.

5. Framework coverage

Every numbered requirement of the source framework, its implementation status, and the artefact that evidences it.

§	Requirement	Status	Implementation	Evidence
1	Two independent paths: knowledge path and exam path	Implemented	src/ingestion + src/publish (knowledge); src/station + src/session (exam). No exam-path module imports an ingestion module.	Module graph; ingestion never appears in the compileStation call tree
2	Nine-stage processing chain, upload through evaluation	Implemented	ingest() -> applyReview() -> planPublication() -> compileStation() -> createSession() -> evaluate()	acceptance.test.ts test 10 walks the whole chain
3	Canonical data model, 14 logical entities	Implemented + extended	All 14 entities present as branded types. Added: KeyPoint weighting and pitfall flag, SelectionReason, ExtractionRun supersession.	src/domain/types.ts; schema/001_init.sql
3	Counts computed from approved occurrences, never incremented	Implemented	Recount views v_examiner_case_counts / v_examiner_question_counts are the authority behind cached columns; MemoryStore.verifyCounts() asserts no drift.	acceptance.test.ts test 4
4.1	Upload boundary: MIME, size, authorization, server-side key, idempotency	Implemented	ingest() enforces size and format before any parsing; contentHash() drives the partial UNIQUE index on knowledge_document.	schema ux_document_content_hash; EngineError FILE_TOO_LARGE / UNSUPPORTED_FORMAT
4.2	Extraction stages with explicit failure states	Implemented	Parser registry, normalization, segmentation, typed candidate extraction, provenance attachment. Each stage has a distinct error code.	src/ingestion/pipeline.ts; EngineErrorCode union
4.2	OCR_REQUIRED rather than fabricated candidates	Implemented	assessTextQuality() gates on letter ratio and mean token length; failure throws before any candidate is constructed.	acceptance.test.ts test 6
4.3	parseDocument pseudocode, optional semantic extractor	Implemented	Grammar extractor is the system of record; CandidateExtractionProvider is the seam for an optional semantic one.	src/ingestion/extractor.ts
4.3	Parse once, never re-parse during an exam	Implemented	No ingestion code is reachable from the session or evaluation paths.	Import graph
5	Per-entity confidence rules (specialty/examiner/case/question/answer)	Implemented	Specialty: controlled alias map. Examiner: two-gate, exact-only auto-merge. Case: qualifier-conflict rule. Question: 9-step pipeline. Answer: never inferred.	src/resolution/*
5	resolveExaminer -> MATCHED NEW_CANDIDATE AMBIGUOUS	Implemented + strengthened	Contract preserved. Added Fellegi-Sunter posterior with per-field log Bayes factors so AMBIGUOUS carries an explanation, plus an EM refiner.	src/resolution/fellegi-sunter.ts
5	Never silently merge two similar examiner names	Implemented	Automatic match requires exact canonical name or registered alias. Every typo variant returns AMBIGUOUS regardless of score.	acceptance.test.ts test 5b: 4 adversarial name pairs, 0 auto-merges
5	Question dedup: 9 steps including optional semantic similarity	Implemented	Steps 1-6 in QuestionDeduplicator; step 5 filled deterministically by concept-set comparison rather than a model.	Cross-language match at lexical similarity 0.000
6	Six candidate states; only PUBLISHED is student-visible	Implemented	Explicit transition table. pathsToPublished() enumerates every route to PUBLISHED for exhaustive assertion.	unit.test.ts SAFETY test
6	Idempotent publication transaction	Implemented	Deterministic occurrence fingerprint + UNIQUE index. Plan/apply split keeps the transaction short.	acceptance.test.ts tests 4 and 10

§	Requirement	Status	Implementation	Evidence
6	Answer curation separate from historical evidence	Implemented	Uncurated source answers publish as candidates but never become an ExpectedAnswer; evaluationReady stays false.	publisher.ts curatedAnswers gate
7	Station compiler with the five-term selection score	Implemented	All five terms with the framework's weights, configurable via StationPolicy.	src/station/compiler.ts DEFAULT_POLICY
7	Deterministic compilation from a stored seed	Implemented	xoshiro128** seeded PRNG; the session row stores seed and policy version.	acceptance.test.ts: identical seeds give byte-identical stations
7	Log the reason each question was selected	Implemented + extended	SelectionReason persisted per session question as a column, not a log line, so it is queryable.	schema session_question.selection_reason
8	Four session phases; server-owned station	Implemented	Explicit phase machine; preparationEndsAt is an absolute timestamp so a refresh cannot extend it.	src/session/session-service.ts
8	Refresh must not regenerate a different station	Implemented	Question order frozen at creation; the view is rebuilt from persisted rows, never recompiled.	acceptance.test.ts reproducibility group
9	Grounded evaluation, seven steps	Implemented	All seven, with ownership checks ordered so a probe cannot distinguish another student's session from a nonexistent one.	acceptance.test.ts test 8
9	Baseline deterministic scoring	Implemented + extended	Coverage rule preserved. Added key-point weighting, hedge discounting, broader-concept partial credit and pitfall penalties.	src/evaluation/evaluator.ts
11	Key points never reach the client before submission	Implemented	The public view is a whitelist, not a redaction. A test asserts no key-point text appears in the serialized payload.	acceptance.test.ts leak test
10.1	Seven named indexing priorities	Implemented	All seven present, plus a covering index for the compiler pool query and a partial index for the pending review queue.	schema/001_init.sql
10.2	No parsing, dedup or heavy joins on the exam-start path	Verified by measurement	Station compilation p95 is 0.105 ms of an 800 ms budget - 0.01%.	bench/latency.bench.ts
10.3	Async ingestion only when latency evidence justifies it	Deferred, deliberately	Ingesting a 10 KB file costs 32 ms p95. Nowhere near a Worker limit; adding a queue now would add failure modes for no measured gain.	bench: ingest p95 32.10 ms
11	Eight risk controls	Implemented	Duplicate publish, wrong merge, in-memory state, secrets, raw file exposure, evaluation abuse, parser corruption, observability.	See risk register
11	Minimum observability event set	Implemented + extended	Closed typed union so a misspelled event name is a compile error. Tag redaction is an allowlist.	src/observability/events.ts
12	Modular monolith with named services	Implemented	Directory structure maps one-to-one onto the framework's recommended module tree.	src/ layout
12	P0/P1/P2 improvements	P0 and P1 complete; P2 seams in place	P0: boundaries, indexes, idempotent fingerprints. P1: versioned extraction, selection-reason logging. P2: provider interfaces exist and are unused.	See roadmap
13	Four service contracts	Implemented	OsceKnowledgeRepository (as KnowledgeSource), StationCompiler, AnswerEvaluationProvider, CandidateExtractionProvider.	src/station/compiler.ts, src/evaluation/evaluator.ts, src/ingestion/extractor.ts

§	Requirement	Status	Implementation	Evidence
14	Ten acceptance tests	Implemented as executable tests	All ten, plus a fifth-and-a-half adversarial examiner-merge test and four reproducibility tests.	test/acceptance.test.ts, 15 tests passing
15	Phase A-D roadmap	Phases A-C complete; D scoped	Baseline measurement, data-integrity hardening and performance hardening are done. Phase D needs a real labelled corpus.	See roadmap

Table 7. Coverage against all fifteen framework sections. “Deferred, deliberately” appears once, for asynchronous ingestion: at 32 ms per file it is nowhere near a Worker limit, and adding a queue now would add failure modes for no measured gain.

6. Competitive analysis

This engine against institutional OSCE platforms, open-source assessment stacks, question banks, and the specialist tooling it borrows from.

Capability	This engine (V2)	Speedwell eSystem	ExamSoft	TAO (OAT)	Moodle Quiz engine	Geeky Medics / OSCEstop	UWorld / AMBOSS	Docling / Unstructured .io	Splink (MoJ)	ETS c-rater / m-rater	LLM-judge ASAG (2024-2026)
Historical examiner attribution as a first-class entity	Full	Partial	Partial	None	None	None	None	N/A	N/A	N/A	N/A
Provenance from student answer back to source file span	Full	Partial	Partial	Partial	Partial	None	None	Partial	N/A	None	None
Ingests unstructured historical recall material	Full	None	None	None	None	None	None	Full	N/A	N/A	Partial
Human review gate before student visibility	Full	Full	Full	Full	Full	Full	Full	None	N/A	N/A	None
Probabilistic entity resolution with explainable weights	Full	None	None	None	None	None	None	None	Full	N/A	None
Zero-tolerance guarantee on wrong identity merges	Full	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Partial	N/A	None
Cross-language semantic answer matching	Full	None	None	None	None	Partial	None	N/A	N/A	Partial	Full
Deterministic, reproducible grading	Full	Full	Full	Full	Full	None	Full	N/A	N/A	Full	None
Free-text answer grading	Full	Partial	Partial	Partial	Partial	Partial	None	N/A	N/A	Full	Full
Negation-aware scoring	Full	None	None	None	None	None	None	N/A	N/A	Partial	Partial
Explains every mark to the student	Full	Partial	Partial	Partial	Partial	Partial	Partial	N/A	N/A	Full	None
Reproducible exam assembly from a stored seed	Full	None	Partial	Partial	Partial	None	None	N/A	N/A	N/A	N/A
Online item difficulty calibration	Full	Partial	Full	Full	Partial	None	Full	N/A	N/A	N/A	N/A
QTI / LTI standards interoperability	None	Partial	Full	Full	Partial	None	None	N/A	N/A	N/A	N/A
Runs at the edge with no server fleet	Full	None	None	None	None	Partial	Partial	None	None	None	None
Zero per-evaluation marginal cost	Full	Full	Full	Full	Full	Partial	Full	Partial	Full	None	None
Institutional circuit / multi-station logistics	None	Full	Full	Partial	None	Full	None	N/A	N/A	N/A	N/A
Examiner tablet marking and offline capture	None	Full	Full	Partial	None	None	None	N/A	N/A	N/A	N/A
Scanned-document OCR	None	N/A	N/A	N/A	N/A	N/A	N/A	Full	N/A	N/A	N/A

Capability	This engine (V2)	Speedwell eSystem	ExamSoft	TAO (OAT)	Moodle Quiz engine	Geeky Medics / OSCEstop	UWorld / AMBOSS	Docling / Unstructured .io	Splink (MoJ)	ETS c-rater / m-rater	LLM-judge ASAG (2024-2026)
Handles paraphrase outside a curated vocabulary	Partial	None	None	None	None	Partial	None	N/A	N/A	Partial	Full

Table 8. Full / Partial / None / N/A, where N/A means the capability is outside that product's category rather than missing from it. Ratings are based on published product documentation; see the sources list.

6.1 Products compared

Product	Category	Type	What it is known for
This engine (V2)	Purpose-built OSCE knowledge engine	In-house	Historical examiner-case-question provenance as the core asset.
Speedwell eSystem	Institutional OSCE delivery	Commercial	Examiner tablet marking, offline app, multi-day circuits, mandatory feedback fields, incomplete-marksheet blocking.
ExamSoft	Institutional assessment + item banking	Commercial	Item banking with category tagging, OSCE case inventories, controlled delivery, psychometric reporting.
TAO (OAT)	Open-source assessment platform	Open source	Only QTI-native open-source authoring, item banking and test assembly; certified in all four QTI categories; LTI integration.
Moodle Quiz engine	LMS assessment subsystem	Open source	question_attempt as an append-only step history; question bank versioning; usage-by-activity abstraction.
Geeky Medics / OSCEstop	OSCE revision content	Commercial / free	1300+ authored stations, checklists, mark schemes, circuit builder, AI virtual patients and AI examiner feedback.
UWorld / AMBOSS	Medical question bank	Commercial	Large authored MCQ banks with explanations, performance analytics and IRT-informed difficulty.
Docling / Unstructured.io	Document ingestion	Open source	Docling: IBM Research layout models, ~97.9% complex-table accuracy, CPU-efficient. Unstructured: 30+ formats, RAG chunking.
Splink (MoJ)	Entity resolution	Open source	Fellegi-Sunter with EM parameter estimation; 1M records in under two minutes on DuckDB; 100M+ on Spark.
ETS c-rater / m-rater	Automated short-answer scoring	Commercial / research	Concept-based scoring against rubric-derived model answers; the reference implementation of key-point ASAG.
LLM-judge ASAG (2024-2026)	Automated short-answer scoring	Research / commercial	GPT-class models with rubric prompting; strong on paraphrase, non-deterministic even at near-zero temperature.

6.2 Reading the matrix

Capability	What the comparison actually shows
Historical examiner attribution as a first-class entity	Peers model examiners as markers, not as the historical identity a question belongs to. This is the differentiating asset.
Provenance from student answer back to source file span	Delivery platforms trace to an item; this traces to the page, line and character offset of the upload that evidenced it.
Ingests unstructured historical recall material	Commercial OSCE platforms assume authored content. Document parsers ingest but do not resolve identity.
Human review gate before student visibility	Parity with institutional platforms; ahead of pure ingestion tooling.
Probabilistic entity resolution with explainable weights	Splink is the reference; this implements the same Fellegi-Sunter model plus a deterministic authority gate.
Zero-tolerance guarantee on wrong identity merges	Splink is purely probabilistic, so its false-merge rate is nonzero by construction. The authority gate makes ours structural.
Cross-language semantic answer matching	Achieved here without a model, via concept identifiers. LLM judges do this too, non-deterministically.

Capability	What the comparison actually shows
Deterministic, reproducible grading	Checklist-based platforms are deterministic because they only score checkboxes. This is deterministic on free text.
Free-text answer grading	Peer platforms grade free text manually or with an LLM. This grades it deterministically against approved key points.
Negation-aware scoring	'No evidence of DVT' must not score as covering DVT. Rare outside clinical NLP research.
Explains every mark to the student	Names the matched key point, the surface form, and the match mechanism.
Reproducible exam assembly from a stored seed	Most platforms store the assembled form; this stores the seed that generates it, which is smaller and replayable.
Online item difficulty calibration	Elo/Rasch online updates; commercial banks use batch IRT, which needs far more responses before it is usable.
QTI / LTI standards interoperability	The clearest gap. TAO is certified in all four QTI categories; this engine exports nothing standard yet.
Runs at the edge with no server fleet	Zero-dependency TypeScript on Workers/D1/R2; peers are JVM, PHP or Python server deployments.
Zero per-evaluation marginal cost	No inference call on any path.
Institutional circuit / multi-station logistics	Out of scope: this is a single-student practice engine, not an exam-day delivery system.
Examiner tablet marking and offline capture	Out of scope for the same reason.
Scanned-document OCR	Detected and refused rather than handled. An OCR adapter is a registry entry, not a redesign.
Handles paraphrase outside a curated vocabulary	The honest limitation. Recall is bounded by vocabulary coverage; misses are reported, not silent.

7. Where it leads, where it trails

An honest reading of the matrix on the preceding pages.

7.1 Genuine differentiation

- **Historical examiner attribution.** Every peer platform models examiners as *markers* - people who score a candidate. This engine models them as the *identity a question belongs to*, which is why examiner resolution gets a two-gate design and a zero-tolerance KPI. No comparable product treats examiner identity as a protected historical asset.
- **Provenance to the character offset.** Delivery platforms trace a mark back to an item. This traces it back to the page, line and character span of the file a student uploaded, through an immutable evidence chain. That is what makes “asked 5 times” defensible rather than decorative.
- **Deterministic free-text grading.** Checklist platforms are deterministic because they only score checkboxes. LLM graders handle free text but are not reproducible. This engine grades free text deterministically, across two languages, with negation awareness - a combination the comparison set does not contain.
- **Reproducible assembly from a stored seed.** Most platforms persist the assembled form. Storing the seed that generates it is smaller, replayable, and makes “why did I get this station” answerable.

7.2 Real gaps

- **No QTI or LTI interoperability.** The clearest gap. TAO is certified in all four QTI categories and ExamSoft integrates broadly; this engine exports nothing standard. The domain model is a superset of QTI's item model, so this is additive work - but until it is done, institutional adoption has an obstacle that has nothing to do with engine quality.
- **No exam-day logistics.** No circuits, no examiner tablets, no offline capture, no multi-day scheduling. This is a deliberate scope boundary rather than a deficiency, but it means the engine complements an institutional platform rather than replacing one.
- **No OCR.** Scanned uploads are detected and refused with OCR_REQUIRED. Refusing is the correct behaviour - the alternative is fabricated candidates - but the coverage gap is real, and closing it is a parser-registry entry rather than a redesign.
- **Paraphrase outside the vocabulary.** Covered at length in section 2.3. This is the price of determinism and it is paid knowingly.

7.3 What was borrowed, and from where

Borrowed from	What	How it was adapted
Splink / Fellegi-Sunter	Probabilistic record linkage with m/u parameters and EM fitting	Same model, plus a deterministic authority gate so automatic merges are exact-only
NegEx / ConText	Rule-based clinical negation and hedge detection	Extended with trigger-position tracking so a key point containing a negation word is not self-negating
UMLS	Surface forms mapped to concept identifiers	Same mechanism at corpus scale, curated by a reviewer rather than a standards body
Moodle question engine	Append-only attempt state and question versioning	Versioned extraction runs that supersede rather than mutate
Elo / Rasch	Online item difficulty calibration	Partial-credit scores rather than binary outcomes; adaptive K factor
Classical test theory	Point-biserial discrimination, Wilson bounds	Surfaces negative-discrimination items as a content-quality work queue
MinHash / SimHash LSH	Sublinear near-duplicate candidate generation	Persisted as band-key rows so dedup is an indexed lookup, not a scan

Table 9. Nothing here is novel in isolation. The engineering contribution is the composition, and the guards added at each seam.

8. Verification

What was tested, what was measured, and what could not be measured yet.

8.1 Acceptance tests

#	Test	Expected invariant	Result	How it is verified
1	Random Pediatrics station	Examiner belongs to specialty; case linked to examiner	PASS	200 randomized compilations; every examiner, case and question re-verified against the published link set
2	Manual examiner	Selected examiner never changes	PASS	100 compilations hold the examiner fixed; a cross-specialty selection raises EXAMINER_SPECIALTY_MISMATCH rather than substituting
3	Pending candidate	Never visible to student repository	PASS	An unlinked question is absent from the pool and from 50 randomized stations
4	Duplicate re-publish	Observation count does not inflate	PASS	Three inserts of an identical fingerprint yield count 1; a different offset is correctly a distinct occurrence
5	Ambiguous examiner	Publication blocked until resolved	PASS	planPublication raises AMBIGUOUS_EXAMINER when a segment has no resolved identity
5b	Adversarial name pairs	No silent merge of similar examiner names	PASS	4 adversarial probes including Hassan/Hussein: 0 auto-merges; exact canonical and registered alias still match
6	Scanned PDF	OCR_REQUIRED; zero fabricated candidates	PASS	CID-encoded garbage raises OCR_REQUIRED before any candidate is constructed
7	Evaluation-ready partial answer	PARTIAL with only approved missing points	PASS	Every reported covered and missing id is checked to be a member of the approved key
8	Invalid session question	Evaluation rejected	PASS	Foreign question, wrong student, inactive phase and double submission all rejected with distinct codes
9	Evaluator failure	Student can self-score and finish	PASS	Questions without an approved key report evaluationReady false and complete under SELF scoring
10	New daily upload	Published knowledge appears with no redeploy	PASS	Full ingest-review-publish walk; replay produces zero new occurrences and recognises every fingerprint

Table 10. The framework's Section 14 invariants as executable tests. Test 5b is an addition: the framework names the Hassan/Hussein problem in prose but does not list a test for it.

8.2 Measured performance

Operation	Corpus	p50	p95	p99	n
Station compilation (random, 5 questions)	7,200 questions / 19,800 occurrences	0.046 ms	0.105 ms	0.176 ms	2000
Answer evaluation (6 key points, mixed languages)	5 representative answers	0.209 ms	0.356 ms	0.581 ms	5000
Question dedup lookup	5,000-question corpus	0.130 ms	1.437 ms	1.992 ms	2000
Examiner resolution (realistic names)	3,000 examiners, 970 blocking buckets	2.057 ms	6.977 ms	8.270 ms	3000
Examiner resolution (degenerate blocking)	3,000 examiners, 4 buckets	17.539 ms	20.252 ms	24.774 ms	300
Ingestion of a 10 KB recall file	40 examiner blocks, 120 questions	21.990 ms	32.103 ms	34.689 ms	200

Table 11. Engine CPU only, Node 22, corpus of 7,200 questions and 19,800 occurrences. Endpoint latency adds database and network time, which this cannot simulate - that is the point of measuring compute separately.

8.3 KPI status

KPI	Target	Measured	Status
Station creation p95	< 800 ms	0.105 ms (engine CPU)	PASS
Evaluation API p95 (deterministic)	< 300 ms	0.356 ms (engine CPU)	PASS
Next question client transition p95	< 150 ms	0 ms engine cost	PASS

KPI	Target	Measured	Status
Upload acknowledgement p95	< 1.5 s	32.1 ms for a 10 KB file	PASS
Admin candidate list p95	< 800 ms	Not measured (I/O bound)	By design
Extraction candidate precision after review	>= 90%	Not yet measurable	Blocked
Incorrect examiner auto-merge	0 tolerated	0	PASS
Station creation failure rate	< 0.5%	0 in 350 compilations	PASS
Duplicate publish incidents	0	0	PASS
Dynamic publication delay after reviewer action	< 5 s	Not measured (I/O bound)	By design
Browser console core errors	0	N/A	Out of scope

Table 12. Two KPIs are marked "by design" because they are I/O-bound and need a live database to measure. One is blocked on a labelled corpus. The rest pass.

A limitation found by measuring rather than assuming

Examiner blocking degrades when many names share a phonetic skeleton. On a realistic Arabic name distribution the index yields 970 buckets with a mean of 16 and a **maximum of 910** - that largest bucket is every examiner whose given name sounds like "Ahmed". Resolution still returns in about 7 ms and it is an admin path, so this is acceptable today. It will not be at ten times the examiner count. The benchmark deliberately includes a degenerate corpus where blocking collapses to four buckets, so the cost of that failure (about 20 ms) is known in advance rather than discovered in production. The fix, when the largest bucket passes roughly 2,000 records, is a compound given+family blocking key - one function.

9. Risk register

Every risk in the framework's Section 11 table, plus the ones this implementation surfaced. Status is Closed only where a test enforces the control.

ID	Risk	Class	Impact	Likelihood	Control	Status	Evidence
R1	Duplicate publish inflates observation counts	Data integrity	High	Low	Deterministic fingerprint + UNIQUE index + plan-level dedup + derived recount views	Closed	acceptance test 4
R2	Wrong examiner merge corrupts the historical asset	Data integrity	Critical	Low	Two-gate resolution; automatic match requires exact canonical or registered alias	Closed	acceptance test 5b
R3	Correctness-critical in-memory state lost on Worker restart	Reliability	High	Low	No engine module holds mutable cross-request state; indexes are built per batch and passed in	Closed	ExaminerResolver.buildIndex is static and returns the index
R4	Approved key points leak to the client before submission	Security	Critical	Low	Public view is a whitelist; a test asserts no key-point text appears in the payload	Closed	acceptance leak test
R5	Evaluation abuse via forged question identifiers	Security	High	Low	Evaluation addressed by sessionQuestionId only; ownership checked before anything is revealed	Closed	acceptance test 8
R6	Silent parser corruption after an extractor change	Data integrity	High	Medium	Versioned extraction runs; reprocessing supersedes rather than mutates and refuses an unchanged version	Closed	reprocess() EXTRACTOR_VERSION_CONFLICT
R7	Vocabulary coverage limits evaluation recall	Content quality	Medium	High	unmatchedTerms reported on every evaluation; confidence score falls as unrecognised terms rise	Managed	The accepted cost of determinism; converted into a reviewer work queue
R8	Examiner blocking degrades as the corpus grows	Performance	Medium	Medium	Measured, not assumed: 970 buckets at 3,000 examiners with a 910-record maximum	Accepted, monitored	Fix is a compound blocking key; one function in phonetic.ts
R9	Thresholds tuned on a small labelled set	Content quality	Medium	High	calibrateThreshold ships with the engine; thresholds are configuration, not constants	Managed	Re-run per corpus; margin 0.198 on the reference set
R10	No QT/LTI export limits institutional adoption	Interoperability	Medium	Medium	Domain model is a superset of QTI's item model; export is additive	Open	Roadmap item P2
R11	Arabic normalization folds a clinically meaningful distinction	Content quality	High	Low	Aggressive folding applies only to comparison keys; stored text uses the conservative level	Closed	normalizeForDisplay / normalizeForMatching separation, idempotence tested
R12	Scanned uploads cannot be processed at all	Coverage	Medium	High	Detected and refused with OCR_REQUIRED rather than silently mis-parsed	Open	OCR adapter is a parser-registry entry

10. Roadmap

The framework's Phase A-D plan, extended. Phases A-C are complete and measured; Phase D is blocked on data that does not exist yet, not on engineering.

Phase	Work	Status	Detail	Why it matters
Phase A	Baseline measurement	Complete	Latency profiled, entity map generated, one document traced end to end, in-memory state audited, structured logging added	bench/latency.bench.ts; acceptance test 10
Phase B	Data integrity hardening	Complete	Unique constraints and stable fingerprints, centralized resolution policy, derived counts, versioned extraction runs	schema/001_init.sql; src/resolution; src/publish
Phase C	Performance hardening	Complete	All Section 10.1 indexes plus covering and partial indexes; no parsing on the exam path; order compiled once; keyset pagination	Measured: 0.01% of the station-creation budget
Phase D.1	Labelled corpus benchmark	Next	Label 300-500 real recall segments; measure extraction precision and recall per field; publish the confusion table	Unblocks the >=90% precision KPI
Phase D.2	Threshold recalibration on real data	Next	Run calibrateThreshold against reviewer merge decisions; pin the operating point in policy	Replaces reference-set thresholds with corpus-fitted ones
Phase D.3	Fellegi-Sunter EM fitting	Next	Replace informed m/u priors with EM estimates once labelled pairs exist; refineWithEm already ships	Narrows the AMBIGUOUS band, reducing reviewer load without weakening the authority gate
Phase D.4	Vocabulary expansion loop	Continuous	Aggregate unmatchedTerms weekly; add recurring terms; track evaluation recall as the metric	The mechanism that lifts the determinism ceiling over time
Phase E.1	OCR parser adapter	Backlog	Register an OCR-backed pdf parser; scanned uploads become ingestible instead of refused	Registry entry, not a redesign
Phase E.2	QTI 3.0 export	Backlog	Export published questions and answer keys as QTI items for institutional interoperability	Closes the clearest gap against TAO and ExamSoft
Phase E.3	Compound blocking key	Backlog, triggered	Switch to a given+family compound key when the largest blocking bucket exceeds ~2,000 records	Trigger condition is measurable from ExaminerIndex.stats
Phase E.4	Optional semantic adapter	Backlog, conditional	Add a semantic provider behind the existing interfaces for the residue the vocabulary misses - never on the critical path	Only after Phase D.1 quantifies what the residue actually is

Table 13. Phases E.3 and E.4 are triggered rather than scheduled: E.3 when the largest examiner blocking bucket exceeds roughly 2,000 records, which the index reports directly; E.4 only if the labelled-corpus benchmark shows the vocabulary's residue justifies a semantic adapter.

11. Recommendations

In the order they should be done.

1. Label a corpus. This is the highest-value next action by a wide margin.

Three to five hundred real recall segments, labelled by a reviewer for examiner, case, question, year and answer. It unblocks the extraction-precision KPI, supplies training pairs for Fellegi-Sunter EM fitting, and lets every threshold in the engine be recalibrated against reality instead of against a reference set of eight pairs. It is a labelling exercise, not an engineering one, and nothing else about content quality can be settled without it.

2. Run the engine against production data before changing anything else.

The thresholds, the vocabulary and the blocking scheme are all configuration. The most likely outcome of the first real run is a list of vocabulary gaps and a threshold adjustment - both cheap. Discovering those after building further on top is not.

3. Wire the observability events to a real sink and watch four numbers.

Evaluation fallback rate, examiner-resolution ambiguity rate, unmatched-term frequency, and station creation p95. The first three are content-quality signals that no amount of code review substitutes for; the fourth confirms the latency claims hold under real I/O.

4. Add QTI 3.0 export when institutional adoption becomes the goal.

Not before. It is real work with no benefit to a single-institution deployment, and the domain model already supports it whenever it is wanted.

5. Add OCR as a parser-registry entry, not as a redesign.

The refusal path is correct and should stay. An OCR-backed pdf parser slots in beside the existing ones and changes nothing downstream, because the text-quality gate will still catch a bad OCR result.

6. Consider a semantic adapter only after step 1 quantifies the residue.

If the labelled corpus shows the vocabulary misses five percent of answers, the adapter is not worth its cost. If it misses thirty, it is. That number does not exist yet, and building the adapter before it does means never learning whether it was needed.

11.1 What this engine is, and is not

It is a knowledge engine with an examination runtime attached. Its differentiating asset is the traceable examiner-case-question graph built from historical recall material, and every design decision that looks conservative - the two-gate resolver, the review boundary, the refusal to OCR, the refusal to infer an answer key - protects that asset.

It is not an exam-day delivery platform and should not become one. Circuits, examiner tablets and invigilation are a different product with different constraints, and the institutions that need them already buy them. The right relationship is complementary: this engine produces the validated question bank, and a delivery platform consumes it - which is precisely the argument for the QTI export in recommendation four.

11.2 Evidence index

Claim	Value	Command
Test suite	93 tests, 0 failures	<code>npm test</code>
Type safety	Clean under strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes	<code>npm run typecheck</code>
Acceptance	15 tests covering all 10 framework invariants plus reproducibility	<code>npm run test:acceptance</code>
Latency	6 operations profiled at p50/p95/p99	<code>npm run bench</code>
Hash correctness	FNV-1a 64 verified against 4 published vectors	<code>npm test</code>

Claim	Value	Command
Metric separation	same-question 0.826-0.946 vs different-question 0.128-0.627	npm test
Threshold calibration	precision 1.0, recall 1.0, margin 0.198 on the reference set	npm test
Cross-language matching	English/Arabic question matched at lexical similarity 0.000 via concept set	manual probe
Merge safety	0 auto-merges across 4 adversarial name pairs	npm run test:acceptance

Table 14. Every quantitative claim in this report is reproducible from the accompanying source.

Sources

Product documentation, standards and research consulted for the comparative analysis.

Source	URL
Speedwell eSystem - OSCE exam software	https://www.speedwellsoftware.com/exam-software/osce-exams/
Speedwell - OSCE marking and feedback	https://www.speedwellsoftware.com/enhance-osces-marking-and-feedback/
TAO - open-source QTI-native assessment platform	https://www.taotesting.com/products/
TAO - QTI compliance in all four categories	https://www.taotesting.com/news/oat-announces-qti-compliance-for-its-tao-open-source-assessment-platform/
Moodle - overview of the question engine	https://docs.moodle.org/dev/Overview_of_the_Moodle_question_engine
Moodle - Questions API	https://moodledev.io/docs/5.0/apis/subsystems/question
1EdTech - QTI v3.0 overview	https://www.imsglobal.org/spec/qti/v3p0/oview
1EdTech - xAPI and Caliper comparison	https://www.imsglobal.org/initial-xapicaliper-comparison
Geeky Medics - OSCE station bank	https://geekymedics.com/osce-stations/
OSCEstop - OSCE revision	https://oscestop.education/
Splink - probabilistic vs deterministic linkage	https://moj-analytical-services.github.io/splink/topic_guides/theory/probabilistic_vs_deterministic.html
Splink - fuzzy matching at scale	https://www.robinlinacre.com/introducing_splink/
Heuristic negation detection for clinical text (NegEx/ConText lineage)	https://pmc.ncbi.nlm.nih.gov/articles/PMC7910278/
DEEPEN - dependency-aware extension of NegEx	https://www.sciencedirect.com/science/article/pii/S153204641500043X
Elo rating in adaptive educational systems	https://www.sciencedirect.com/science/article/abs/pii/S036013151630080X
On-the-fly IRT parameter estimation in adaptive learning	https://link.springer.com/article/10.3758/s13428-022-01953-x
Automatic short answer grading in the LLM era	https://dl.acm.org/doi/full/10.1145/3706468.3706481
Quality-conditioned agreement in automated short answer scoring	https://arxiv.org/pdf/2605.07647
Docling - advanced layout analysis models	https://arxiv.org/pdf/2509.11720
Cloudflare Workers - platform limits	https://developers.cloudflare.com/workers/platform/limits
Anki - spaced repetition algorithm (FSRS)	https://faqs.ankiweb.net/what-spaced-repetition-algorithm